Yeditepe University
Department of Computer Engineering

CSE 232 Systems Programming
Spring 2026

Term Project

Due to: 8 June 2026

5 students in a group

In this project, you will develop a simple **text editor with garbage collection**.

Write your text editor in C and use gcc compiler on Linux.

The editor has a simple user interface, manages the text buffer in the memory and does garbage collection.

User Interface

The editor must have the following functions:
- E (edit): Opens the specified file and reads it into the text buffer
- I (insert): Inserts the line in text buffer
- D (delete): Deletes the line
- R (replace): Replaces the character
- P (print): Displays the text on the screen
- S (save): Saves the file
- G (garbage collection): to start garbage collection
- Q (quit): quits editor

The editor must have a user interface that displays the text on the screen. Each statement of the test is displayed on a separate line. The user must be able to select a line by moving the cursor on the screen. To insert or delete a statement, the user first displays the text on the screen using P command. Then moves the cursor up/down with the ↑ and ↓ keys and presses ↵ key to select the line.

- To delete the selected line, the user presses D key.
- To insert a new statement in the text, after selecting a line using ↑ ↓ ↵ keys, the user presses I key, and writes the new statement from the keyboard. The new statement is inserted **after** the selected line.
- To replace a character, after selecting the line using ↑ ↓ ↵ keys, the user moves the cursor to the character to be replaced using → and ← keys, presses R and writes the new character.

Use NCURSES library to implement the cursor functions.

Editor Functions and Data Structure

The editor must keep the text in the memory in a text buffer using the following data structure:

```
struct node {
       char   statement[40];  // max. 40 characters
       int    next;         // points to the textbuffer[] index of the next statement
       int    prev;         // points to the textbuffer[] index of the previous statement
}

struct node textbuffer[100]; // max. 100 lines in the text buffer
int head;     // points to the first valid statement in the "next" link of textbuffer[]
int tail;     // points to the first valid statement in the "prev" link of textbuffer[]
int free;     // points to the first free line at the end of the textbuffer[]
```

`textbuffer[100], head, tail` and `free` are global.

Write the following editing functions:

`edit(*char filename)`
Opens the specified file (with .txt extension), reads the text, stores it in `textbuffer[]` and sets the links.

`int cursorLine()`
Using the cursor coordinates on the screen, it calculates and returns the `textbuffer[]` index of the current line.

`int cursorChar()`
Using the cursor coordinates on the screen, it calculates and returns the `textbuffer[]` character position of the cursor on the selected line.

`delete(int index)`
Parameter `index` is the location of the cursor, returned by `cursorLine()` function. This function updates the links in order to remove access to the deleted statement. Then the text is refreshed on the screen.

`insert(int index)`
Parameter `index` is the location of the cursor after which the new line will be inserted (returned by `cursorLine()` function). This function reads the new statement from the keyboard, stores it at the end of `textbuffer[]` and updates the links. Then the text is refreshed on the screen.

`replace(int index)`
Parameter `index` is the character position of the cursor, returned by `cursorChar()` function. This function updates the character in the `textbuffer[]`.

`print()`
Used to refresh the text on the screen. Starting from the head of the `textbuffer[],` follows the links, and displays the text. Assume that maximum text size is 30 lines.

`save()`
Starting from the head of the `textbuffer[],` following the links, writes the text in the .txt file and closes the file.

Garbage Collection

Garbage collection can either be started by the user using G command or it is done automatically. Automatic garbage collection should start if there is no space left at the end of the `textbuffer[]` for insertions or after every 10 insertions/deletions. Write the following function:

`int garbageCollection()`
Shift all valid data in the `textbuffer[]` in order to utilize the deleted entries (the entries that do not have any access through the links). At the end, all unused entries will be at the end of the `textbuffer[]`, available for new insertions.

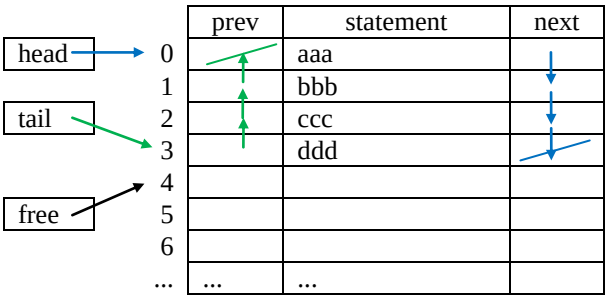
Example:

mytext.txt
aaa
bbb
ccc

Open for editing using “E mytext.txt” command

```
aaa
bbb
ccc
ddd
>
```

`Textbuffer[]` after edit command



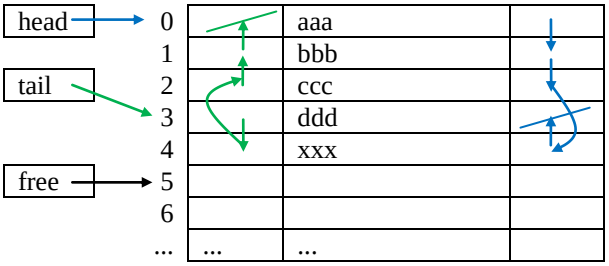
Insertion: Move the cursor, press I and write the line

```
aaa
bbb
ccc
ddd
>
```

```
aaa
bbb
ccc
ddd
>xxx
```

```
aaa
bbb
ccc
xxx
ddd
>
```

`Textbuffer[]` after insertion

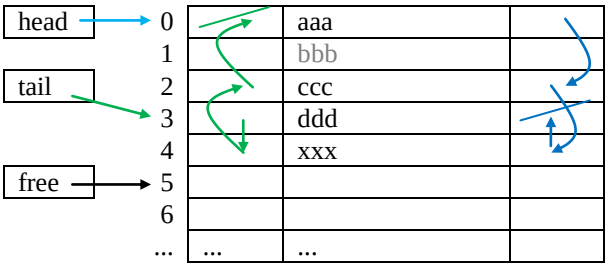


Deletion: Move the cursor and press D

```
aaa
bbb
ccc
xxx
ddd
>
```

```
aaa
ccc
xxx
ddd
>
```

`Textbuffer[]` after deletion



Replacement: Move the cursor, press R and write the new character

```
aaa
ccc
xxx
ddd
>
```

```
aaa
ccc
xxy
ddd
>
```

Garbage collection:

`Textbuffer[]` after garbage collection